

Relazione Tecnica — Sistema Informativo Gestione Ospedaliera

Gruppo09 — Enrico Muselli, Ferdinando Longobardo, Francesco Megna

Indice

1. Introduzione
2. Architettura del Sistema
3. Pattern BCE + DAO
4. Il Package Model
 - 4.1 Utente
 - 4.2 Admin
 - 4.3 Medico
 - 4.4 Paziente
 - 4.5 Reparto
 - 4.6 Stanza
 - 4.7 Letto
 - 4.8 Ricovero
 - 4.9 Prestazione
 - 4.10 Turno
 - 4.11 AssenzaMedico
5. Associazioni tra le classi del Model
6. Il Package DAO
7. Il Package ImplementazioneDao
8. Il Package database_connection
9. Il Package Controller

10. Il Package GUI
11. Il Database PostgreSQL
12. Documentazione
13. Scelte Architetture e Motivazioni

1. Introduzione

Il progetto consiste nello sviluppo di un sistema informativo per la gestione di una struttura ospedaliera. Il sistema è composto da una base di dati relazionale PostgreSQL e da un applicativo Java dotato di interfaccia grafica realizzata con la libreria Swing.

Il software permette di gestire pazienti, ricoveri, turni del personale medico e prestazioni cliniche, garantendo la coerenza dei dati e il rispetto dei vincoli operativi descritti nella traccia.

2. Architettura del Sistema

Il sistema adotta il pattern architetturale **BCE + DAO** (Boundary-Control-Entity + Data Access Object), che estende il classico pattern BCE aggiungendo uno strato dedicato alla persistenza dei dati.

I package del progetto sono sei, ciascuno con responsabilità ben definite e separate:

- **model** — contiene le classi di dominio con i dati e la logica in memoria
- **controller** — coordina le richieste tra GUI e strato di accesso ai dati
- **dao** — contiene le interfacce che descrivono i metodi di accesso al database
- **implementazioneDao** — contiene le implementazioni concrete per PostgreSQL
- **database_connection** — gestisce la connessione al database tramite il pattern Singleton
- **gui** — gestisce l'interfaccia utente con la libreria Swing

Il flusso delle dipendenze è sempre unidirezionale: la GUI chiama solo il Controller, il Controller chiama Model e DAO, il Model non conosce l'esistenza del DAO. Questa separazione garantisce che ogni strato possa essere modificato o sostituito senza impattare gli altri.

3. Pattern BCE + DAO

La scelta di BCE + DAO invece della variante BCED è motivata da ragioni di efficienza. Nella soluzione BCED il Model chiama direttamente il database per qualsiasi operazione, anche per dati già presenti in memoria. Il database è ordini di grandezza più lento della memoria e può essere usato in concorrenza da più utenti, rendendo BCED una soluzione inefficiente.

Con BCE + DAO il Controller decide autonomamente quando accedere al Model in memoria e quando interrogare il database tramite DAO. I dati vengono letti dal database una volta e mantenuti in memoria, tornando al database solo quando serve persistere una modifica.

Il vantaggio aggiuntivo del pattern DAO è la separazione tra interfaccia e implementazione. Il package dao contiene solo interfacce che descrivono cosa fare sul database, mentre il package implementazioneDao descrive come farlo per PostgreSQL. Se in futuro si volesse cambiare database sarebbe sufficiente aggiungere un nuovo package di implementazione senza toccare nulla del resto del sistema.

4. Il Package Model

Il Model contiene le classi di dominio del sistema. Seguendo il pattern BCE + DAO, nessuna classe del Model importa nulla dal package dao o implementazioneDao. Il Model contiene esclusivamente dati e logica che opera in memoria.

4.1 Utente

Superclasse della gerarchia degli utenti. Contiene i dati di autenticazione comuni a tutte le tipologie di utente.

Attributi:

- `username` — nome utente univoco per l'accesso al sistema
- `password` — password associata all'account

Metodi:

- `mostraInfo()` — metodo di utilità per il debug, sovrascritto dalle sottoclassi

4.2 Admin

Sottoclasse di Utente che rappresenta il personale amministrativo. Non aggiunge attributi propri. Esiste come tipo distinto per consentire al Controller di verificare i permessi tramite `instanceof Admin`.

4.3 Medico

Sottoclasse di Utente che rappresenta il personale medico.

Attributi aggiuntivi:

- `matricola` — codice identificativo univoco del medico

- `reparto` — reparto di appartenenza

Metodi:

- `appartieneAREparto(int idReparto)` — controlla in memoria se il medico appartiene al reparto indicato

4.4 Paziente

Rappresenta un paziente registrato nel sistema.

Attributi:

- `codiceFiscale` — identificativo univoco del paziente
- `nome, cognome` — dati anagrafici

Metodi:

- `getInfoBase()` — restituisce una stringa riassuntiva nel formato "Nome Cognome - CF", utile per le liste della GUI

4.5 Reparto

Rappresenta un reparto della struttura ospedaliera.

Attributi:

- `idReparto` — identificativo univoco
- `nome` — denominazione del reparto
- `stanze` — lista delle stanze del reparto
- `medici` — lista dei medici afferenti

Metodi:

- `aggiungiMedico(Medico medico)` — aggiunge un medico al reparto e aggiorna il riferimento bidirezionale impostando il reparto sul medico
- `aggiungiStanza(Stanza stanza)` — aggiunge una stanza al reparto
- `cercaLettiLiberi()` — scorre tutte le stanze e raccoglie i letti non occupati

4.6 Stanza

Rappresenta una stanza all'interno di un reparto.

Attributi:

- `numero` — identificativo della stanza nel reparto

- `capienza` — numero massimo di letti
- `letti` — lista dei letti contenuti

Metodi:

- `aggiungiLetto(Letto letto)` — aggiunge un letto alla stanza
- `getLettiDisponibili()` — restituisce i letti con stato occupato uguale a false

4.7 Letto

Rappresenta un posto letto. Il codice è univoco nell'intero ospedale, non solo nel reparto o nella stanza.

Attributi:

- `idLetto` — codice univoco, corrisponde a `codiceletto` nel database
- `occupato` — stato calcolato a runtime dal DAO tramite query SQL

4.8 Ricovero

Rappresenta l'associazione tra un paziente e un letto per un determinato periodo. Contiene sia le date di dimissione previste che quelle effettive.

Attributi:

- `idRicovero` — identificativo univoco generato dal database con SERIAL
- `dataInizio, oraInizio` — inizio del ricovero
- `dataDimissionePrevista, oraDimissionePrevista` — dimissione pianificata
- `dataDimissioneEffettiva, oraDimissioneEffettiva` — dimissione reale, null finché il paziente non viene dimesso
- `paziente` — paziente ricoverato
- `letto` — letto assegnato

Metodi:

- `inCorso()` — controlla in memoria se `dataDimissioneEffettiva` è null
- `inScadenzaIl(LocalDate data)` — controlla se la dimissione prevista cade nella data indicata

4.9 Prestazione

Rappresenta un intervento medico, visita o chirurgico, eseguito durante un ricovero.

Attributi:

- `idPrestazione` — identificativo univoco generato dal database
- `tipo` — tipologia della prestazione
- `data` — data in cui viene eseguita (campo aggiunto durante lo sviluppo per correggere un bug dell'agenda settimanale che usava erroneamente la data del ricovero)
- `oraInizio, oraFine` — fascia oraria
- `esito` — referto compilato dal medico, inizialmente null
- `medico` — medico che esegue la prestazione
- `ricovero` — ricovero associato

Metodi:

- `checkSovrapposizione(Prestazione altra)` — confronto in memoria tra due prestazioni nella stessa giornata
- `modificaEsito(String nuovoEsito)` — aggiorna l'esito in memoria; la persistenza è delegata al Controller tramite DAO

4.10 Turno

Rappresenta una fascia oraria lavorativa settimanale assegnata a un medico.

Attributi:

- `idTurno` — identificativo univoco
- `oraInizio, oraFine` — fascia oraria del turno
- `giornoSettimana` — giorno della settimana nel formato italiano (Lunedì, Martedì ecc.)
- `medico` — medico assegnato

Metodi:

- `copreFascia(LocalTime ora)` — verifica se un orario rientra nel turno
- `copreFasciaCompleta(LocalTime inizio, LocalTime fine)` — verifica se una fascia oraria è interamente coperta dal turno

4.11 AssenzaMedico

Rappresenta un periodo di assenza per malattia di un medico.

Attributi:

- `dataInizio`, `dataFine` — periodo di assenza
- `medico` — medico assente

Metodi:

- `copreData(LocalDate data)` — controlla in memoria se una data rientra nel periodo di assenza
-

5. Associazioni tra le classi del Model

Utente → **Admin e Medico** — relazione di generalizzazione. Admin e Medico ereditano username e password da Utente.

Medico → **Reparto** — associazione molti a uno. Un medico afferisce a esattamente un reparto, un reparto ha uno o più medici. L'associazione è bidirezionale — il Reparto mantiene la lista dei medici e ogni Medico ha un riferimento al proprio Reparto.

Medico → **AssenzaMedico** — associazione uno a molti. Un medico può avere zero o più periodi di assenza registrati nel tempo.

Medico → **Turno** — associazione uno a molti. Un medico ha uno o più turni lavorativi settimanali.

Medico → **Prestazione** — associazione uno a molti. Un medico esegue zero o più prestazioni nel tempo.

Reparto → **Stanza** — composizione uno a molti. Una stanza non esiste al di fuori di un reparto. Se il reparto viene rimosso le stanze non hanno più senso nel sistema.

Stanza → **Letto** — composizione uno a molti. Un letto esiste solo all'interno di una stanza.

Paziente → **Ricovero** — associazione uno a molti. Un paziente può avere zero o più ricoveri nel tempo.

Letto → **Ricovero** — associazione uno a molti. Un letto è associato a zero o più ricoveri nel tempo, non sovrapposti.

Ricovero → **Prestazione** — associazione uno a molti. Durante un ricovero possono essere eseguite zero o più prestazioni.

Turno → **Prestazione** — associazione uno a molti. Un turno può contenere zero o più prestazioni.

6. Il Package DAO

Il package dao contiene esclusivamente interfacce Java. Non contiene codice eseguibile — descrive solo quali operazioni il database deve saper fare, senza specificare come. Questo è il punto di forza del pattern DAO: cambiare database significa aggiungere un nuovo package di implementazione senza toccare le interfacce.

UtenteDAO

- `verificaLogin(username, password)` — controlla le credenziali e restituisce il ruolo come stringa
- `getUtenteByUsername(username)` — restituisce l'oggetto Utente completo

PazienteDAO

- `inserisciPaziente(paziente)` — inserisce o aggiorna tramite ON CONFLICT
- `getAllPazienti()` — lista di tutti i pazienti
- `getPazientiInScadenza(data)` — pazienti con dimissione prevista nella data indicata

RicoveroDAO

- `checkSovrapposizione(ricovero)` — verifica sovrapposizioni sul letto
- `inserisciRicovero(ricovero)` — salva il ricovero tramite procedura SQL
- `getRicoveriAttiviPerComboBox()` — ricoveri attivi formattati per la GUI
- `getRicoveriInScadenza(data)` — ricoveri in scadenza nella data indicata

MedicoDAO

- `getAllMedici()` — lista di tutti i medici
- `getSostitutiIdonei(matricolaAssente, inizio, fine)` — medici disponibili come sostituti
- `verificaDisponibilita(matricola, data, oraInizio, oraFine)` — verifica turno e assenza di sovrapposizioni

PrestazioneDAO

- `getAgendaGiornaliera(matricola)` — prestazioni di oggi per il medico
- `getAgendaSettimanale(matricola)` — prestazioni dei prossimi 7 giorni
- `registraPrestazione(...)` — registra con controlli di turno e sovrapposizione

- `aggiornaEsito(idPrestazione, nuovoEsito)` — aggiorna il referto tramite procedura SQL

LettoDAO

- `getAllLetti()` — tutti i letti con stato occupato calcolato
- `getLettiPerReparto(idReparto)` — letti di un reparto con stato occupato

AssenzaMedicoDAO

- `inserisciAssenza(codMedico, dataInizio, dataFine)` — registra l'assenza tramite procedura
 - `contaRicoveriAttivi(idReparto)` — conta i ricoveri attivi tramite funzione SQL
-

7. Il Package ImplementazioneDao

Ogni classe implementa una delle interfacce del package dao e contiene le query SQL reali per PostgreSQL. Tutte usano PreparedStatement con parametri punto interrogativo per evitare SQL injection. Tutte usano il try-with-resources per garantire la chiusura automatica di PreparedStatement e ResultSet.

UtentePostgresDAO

La query di login cerca username e password nella tabella utente e restituisce tipoutente in minuscolo. Il metodo `getUtenteByUsername` fa una JOIN tra utente, medico e reparto e istanzia polimorficamente Admin o Medico in base al tipo trovato.

PazientePostgresDAO

Il metodo `inserisciPaziente` chiama la procedura SQL `inserisci_paziente` tramite CallableStatement. `getPazientiInScadenza` fa una JOIN tra paziente e ricovero filtrando sulla data di dimissione prevista e sui ricoveri non ancora dimessi.

RicoveroPostgresDAO

Il metodo `checkSovrapposizione` usa COALESCE per gestire sia i ricoveri con dimissione effettiva che quelli ancora attivi, e aggiunge `idricovero <> ?` per non bloccare la modifica di un ricovero esistente. `inserisciRicovero` chiama la procedura SQL `inserisci_ricovero` tramite CallableStatement.

MedicoPostgresDAO

Il metodo `getSostitutildonei` funziona in tre passi: recupera i colleghi dello stesso reparto, per ogni giorno del periodo recupera i turni del medico assente, e per ogni turno chiama `verificaDisponibilita` sul candidato. Il metodo privato `convertiGiorno` traduce `DayOfWeek` nel nome italiano del giorno usato nel database. `verificaDisponibilita` fa tre controlli separati: presenza del turno, assenza di prestazioni sovrapposte, assenza di periodi di malattia registrati.

PrestazionePostgresDAO

Il metodo `registraPrestazione` esegue due controlli prima dell'inserimento — verifica che la fascia oraria rientri nel turno del medico e che non ci siano prestazioni sovrapposte nella stessa data. `aggiornaEsito` chiama la procedura SQL `aggiorna_esito_prestazione` tramite `CallableStatement`.

LettoPostgresDAO

Entrambi i metodi calcolano lo stato occupato direttamente in SQL tramite una subquery `EXISTS` che cerca ricoveri attivi sullo stesso letto, invece di lasciarlo sempre a `false`.

AssenzaMedicoPostgresDAO

`inserisciAssenza` chiama la procedura SQL `inserisci_assenza` tramite `CallableStatement`. `contaRicoveriAttivi` chiama la funzione SQL `conta_ricoveri_attivi` tramite `PreparedStatement` con `SELECT`, non con `CALL`, perché è una funzione che restituisce un valore e non una procedura.

8. Il Package `database_connection`

ConnessioneDatabase

Implementa il pattern Singleton per garantire che esista al massimo una sola connessione attiva durante tutta l'esecuzione. Il costruttore è privato per impedire istanziazioni esterne. Il metodo `getInstance` controlla se l'istanza esiste e se la connessione è ancora aperta prima di restituirla, creandone una nuova solo se necessario. Il driver JDBC viene caricato dinamicamente con `Class.forName` per facilitare un eventuale cambio di database in futuro.

Le credenziali sono scritte direttamente nel codice per semplicità didattica .

9. Il Package Controller

Controller

È il coordinatore centrale del sistema. Viene istanziato una volta sola nel Main e passato a tutte le finestre. Tutti e sette i DAO sono attributi finalizzati nel costruttore — non vengono mai ricreati durante l'esecuzione, evitando di aprire connessioni inutili al database.

Mantiene in memoria l'utente loggato tramite l'attributo `utenteLoggato`. Ogni metodo operativo verifica prima il tipo dell'utente loggato con `instanceof` per garantire che solo il ruolo corretto possa eseguire l'operazione.

Metodi di autenticazione:

- `login(username, password)` — verifica le credenziali e carica l'oggetto utente completo
- `logout()` — azzera `utenteLoggato`
- `getUtenteLoggato()` — restituisce l'utente corrente

Metodi amministratore:

- `gestisciPazienti(paziente)` — inserisce o aggiorna tramite DAO
- `gestisciRicoveri(ricovero)` — verifica sovrapposizione e registra
- `calcolaSostituti(matricolaAssente, inizio, fine)` — delega al DAO
- `getPazientiInScadenzaOggi()` — usa `LocalDate.now()`
- `getPazientiInScadenza(data)` — per una data specifica
- `getLettiPerReparto(idReparto)` — letti con stato aggiornato
- `inserisciAssenza(codMedico, dataInizio, dataFine)` — registra l'assenza
- `contaRicoveriAttivi(idReparto)` — chiama la funzione SQL

Metodi medico:

- `agendaGiornaliera()` — estrae la matricola dall'utente loggato
- `agendaSettimanale()` — stessa logica
- `registraPrestazione(...)` — usa `LocalDate.now()` per la data
- `aggiornaEsitoPrestazione(idPrestazione, nuovoEsito)` — aggiorna il referto

Metodi di lettura per la GUI:

- `recuperaTuttiPazienti(), recuperaTuttiLetti(), recuperaTuttiMedici(), recuperaRicoveriPerComboBox()` — popolano le ComboBox e le tabelle
-

10. Il Package GUI

Le classi GUI gestiscono esclusivamente la presentazione visiva. Non contengono logica di business — al massimo validazione formale degli input come campi vuoti o formato data errato. Ogni interazione è delegata al Controller con una singola chiamata a metodo.

Ogni pannello riceve nel costruttore il riferimento al Controller e al frameChiamante. Quando apre una nuova finestra nasconde se stessa e quando chiude ripristina il chiamante con `setVisible(true)` e chiama `dispose()` per liberare memoria.

LoginPanel

Raccoglie username e password, chiama `controller.login` e smista verso `AdminPanel` o `MedicoPanel` in base al ruolo restituito. In caso di credenziali errate mostra un `JOptionPane` di errore.

AdminPanel

Dashboard amministratore con sei pulsanti: gestione pazienti, gestione ricoveri, sostituzioni medici, dimissioni, letti per reparto, registra assenza. Implementa il logout che azzera la sessione e ripristina il login.

MedicoPanel

Dashboard medico con tre pulsanti: agenda giornaliera, agenda settimanale, registra prestazione. Logout identico all'`AdminPanel`.

PazientePanel

Form con tre campi — codice fiscale, nome, cognome. Al salvataggio crea un oggetto `Paziente` e lo passa al Controller.

RicoveroPanel

Form con `ComboBox` paziente e letto, e campi per date e ore. Il `pack()` viene chiamato dopo il popolamento delle `ComboBox` per calcolare correttamente le dimensioni. Al salvataggio costruisce un oggetto `Ricovero` minimo con i soli dati necessari e lo passa al Controller.

AgendaGiornalieraPanel

Tabella con le prestazioni di oggi. Ha il pulsante `Modifica Esito` che apre un `JOptionPane.showInputDialog` precompilato con l'esito attuale e chiama `controller.aggiornaEsitoPrestazione`. La tabella si ricarica dopo ogni modifica.

AgendaSettimanalePanel

Tabella con le prestazioni dei prossimi sette giorni. Usa `p.getData()` per la colonna data, che è il campo diretto della prestazione e non la data del ricovero.

RegistraPrestazionePanel

ComboBox con i ricoveri attivi formattati, campi tipo e orari. La data viene impostata automaticamente a oggi dal Controller tramite `LocalDate.now()` senza che l'utente debba inserirla.

SostituzioniPanel

ComboBox medici assenti, campi date periodo, tabella risultati. Estrae la matricola dal testo formattato della ComboBox con `split`.

DimissioniPanel

Due pulsanti — Dimissioni Oggi e Cerca per Data. Popola una tabella con i pazienti in scadenza recuperati dal Controller.

LettiRepartoPanel

Campo ID reparto e tabella risultati. I letti occupati appaiono in rosso grazie a un `DefaultTableCellRenderer` personalizzato che imposta `setForeground(Color.RED)`. Il titolo della finestra si aggiorna con il conteggio dei ricoveri attivi calcolato dalla funzione SQL.

AssenzaPanel

ComboBox medici e campi date. Registra il periodo di assenza tramite Controller che chiama la procedura SQL.

Main

Entry point dell'applicazione. Istanza il Controller una volta sola, crea il `JFrame` principale, inserisce il `LoginPanel` e lo rende visibile.

11. Il Database PostgreSQL

Schema

Le tabelle principali sono undici: `utente`, `admin`, `medico`, `reparto`, `stanza`, `letto`, `paziente`, `ricovero`, `turno`, `assenza_medico`, `prestazione`. Tutte le chiavi primarie numeriche sono definite come `SERIAL` e vengono generate automaticamente dal database — il codice Java non le passa mai nelle `INSERT`.

Vincoli aggiuntivi

Oltre alle chiavi primarie, alle chiavi esterne e ai vincoli NOT NULL, sono stati aggiunti otto vincoli semantici motivati dalla traccia:

- `chk_tipoutente` — tipoutente può contenere solo ADMIN o MEDICO
- `chk_turno_ore` — orafine deve essere maggiore di orainizio nel turno
- `chk_prestazione_ore` — stessa regola per la prestazione
- `chk_ricovero_date` — la dimissione prevista non può essere prima dell'inizio
- `chk_dimissione_effettiva` — la dimissione effettiva non può essere prima dell'inizio
- `chk_assenza_date` — la data fine assenza non può essere prima della data inizio
- `chk_cf_lunghezza` — il codice fiscale deve essere esattamente 16 caratteri
- `chk_capienza_positiva` — la capienza della stanza deve essere maggiore di zero

Procedure SQL

Cinque procedure sono state definite per centralizzare la logica di inserimento nel database:

inserisci_paziente — gestisce sia l'inserimento che l'aggiornamento in una sola operazione tramite ON CONFLICT DO UPDATE. Se il codice fiscale esiste già aggiorna nome e cognome invece di dare errore.

inserisci_ricovero — riceve tutti i dati del ricovero e lo inserisce. L'id viene generato automaticamente dal database.

inserisci_assenza — prima di inserire controlla che la data fine non sia precedente alla data inizio, altrimenti lancia un'eccezione con messaggio descrittivo.

aggiorna_esito_prestazione — aggiorna il campo esito e gestisce il caso in cui l'id non esiste con NOT FOUND.

Funzione SQL

conta_ricoveri_attivi(idReparto) — conta i ricoveri attivi in un reparto facendo una JOIN tra ricovero, letto e stanza. È stata implementata come funzione invece che procedura perché restituisce un valore direttamente con RETURN, e viene chiamata con SELECT invece di CALL. Viene usata nel pannello LettiRepartoPanel per mostrare il conteggio nel titolo della finestra.

Trigger

Tre trigger garantiscono la coerenza dei dati anche in caso di accesso diretto al database senza passare dall'applicazione Java, aggiungendo un secondo livello di protezione rispetto ai controlli nei DAO.

trg_check_sovrapposizione_ricovero — si attiva BEFORE INSERT OR UPDATE su ricovero. Usa COALESCE per calcolare la data di fine effettiva del ricovero esistente — se è stata dimessa usa quella effettiva, altrimenti quella prevista. Se trova sovrapposizioni lancia un'eccezione che blocca l'operazione.

trg_check_prestazione_turno — si attiva BEFORE INSERT su prestazione. Calcola il giorno della settimana dalla data tramite EXTRACT e una serie di IF/ELSIF, poi verifica che esista un turno del medico che copra la fascia oraria della prestazione. Se non trova il turno blocca l'inserimento.

trg_check_sovrapposizione_prestazione — si attiva BEFORE INSERT OR UPDATE su prestazione. Verifica che il medico non abbia già un'altra prestazione nella stessa data con orario sovrapposto.

12. Documentazione

Il codice è documentato con Javadoc su tutte le classi e i metodi pubblici, con i tag `@author`, `@version`, `@param` e `@return`. La documentazione HTML è stata generata con Tools → Generate Javadoc in IntelliJ con encoding UTF-8 per gestire correttamente i caratteri italiani.

13. Scelte Architetture e Motivazioni

Perché BCE + DAO invece di BCED — in BCED il Model chiama sempre il database anche per dati già in memoria, creando inefficienze. Con BCE + DAO il Controller decide quando usare la memoria e quando il database, ottimizzando le prestazioni e riducendo il numero di query.

Perché i controlli di business sono nel DAO e non nel Controller — i controlli di turno e sovrapposizione in `registraPrestazione` richiedono query al database per funzionare e non possono essere fatti in memoria. Spostarli nel Controller avrebbe richiesto metodi DAO aggiuntivi e aumentato la complessità senza benefici reali. È una semplificazione consapevole rispetto allo schema puro.

Perché il Singleton per la connessione — garantisce che esista al massimo una sola connessione attiva prevenendo problemi di concorrenza. I DAO vengono istanziati una volta sola nel costruttore del Controller per evitare di aprire e chiudere connessioni inutilmente ad ogni chiamata.

Perché i trigger oltre ai controlli Java — i trigger aggiungono un secondo livello di protezione che funziona anche in caso di accesso diretto al database senza passare dall'applicazione. La coerenza dei dati è garantita indipendentemente da come avviene l'accesso.

